



Department of Computer Science and Engineering
Faculty of Automatic Control and Computer Science
National University of Science and Technology
POLITEHNICA Bucharest



Bachelor Thesis

Information-Theoretic Secure Communication over a Simulated QKD Network

Supervisor
Assist. Prof. PhD Eng. Ștefan-Dan
Ciocîrlan

Student
Florin-Teodor Ionița

2026

Contents

Abstract	1
1. Introduction	1
1.1 Motivation	1
1.2 Problem statement	2
1.3 Contribution	2
1.4 The threat model	3
1.5 Outline	4
2. Background	4
2.1 The BB84 protocol	4
2.2 Why eavesdropping causes errors	5
2.3 The Shor–Preskill abort threshold	5
2.4 Information reconciliation: Cascade	5
2.5 Quantum entropy bounds in detail	6
2.6 Privacy amplification	7
2.7 One-Time Pad	7
2.8 Wegman–Carter authentication	7
2.9 Worked example: Cascade on a 16-bit string	8
2.10 TLS 1.3 and the HKDF key schedule	9
2.11 ETSI GS QKD 014	10
3. Architecture	10
3.1 Layering	10
3.2 Message flow	11
3.3 Why two QKD producers	11
3.4 Why ETSI GS QKD 014	12
3.5 Comparison to classical TLS architecture	12
3.6 Failure modes and recovery	13
3.7 Multi-node relay	13
4. Implementation	13
4.1 The visual BB84 simulator	14
4.2 The abstract bulk generator	15
4.3 The C Cascade kernel	15
4.4 Privacy amplification	16
4.5 The Key Management Entity	16
4.6 The QTLS handshake	17
4.7 The QTLS record layer	17
4.8 Rate-limiter analysis	18
4.9 BB84 with an active Eve	18
4.10 The KME store and access control	19
4.11 The QTLS record layer in detail	19
4.12 The visualisation	20
4.13 Testing strategy	21

4.14 Dependencies and footprint	21
5. Evaluation	22
5.1 Experimental setup	22
5.2 Secret-key rate as a function of QBER	22
5.3 Handshake latency	23
5.4 Sustained chat throughput	24
5.5 Re-handshake cadence	25
5.6 Distance / rate sweep	25
5.7 Cascade leakage vs. Shannon bound	25
5.8 Re-handshake stress test	26
5.9 End-to-end throughput	26
5.10 Failure-mode catalog	27
6. Discussion	27
6.1 The SHAKE-128 extractor	27
6.2 Trusted-node relay	28
6.3 Authentication of the classical channel	28
6.4 Side channels	28
6.5 Comparison to ML-KEM	28
6.6 Comparison to commercial QKD stacks	29
6.7 Quantum random number generation	29
6.8 Authentication of the BB84 sifting phase revisited	30
6.9 Comparison to OpenQKD reference implementations	30
6.10 Limitations summary	30
7. Conclusions	31
7.1 Reflections on methodology	32
7.2 Closing remarks	32
References	32
Appendices	35
A.1 Source tree	35
A.2 Reproducing the experiments	36
A.3 Hardware and software environment	36
A.4 Build and dependency notes	37
A.5 Mathematical derivations	37
A.6 Defense demo runbook	38

Abstract

This thesis lies in the field of network security and quantum cryptography. We focus on information-theoretic secure communication protocols that draw their session keys from a Quantum Key Distribution (QKD) infrastructure rather than from classical public-key key-agreement. Currently, deployed TLS stacks rely on Diffie–Hellman key exchange and AES-GCM, both of which are computationally secure: a sufficiently capable quantum adversary can, in principle, break them. Using QKD-derived keys with a One-Time Pad and Wegman–Carter authentication yields unconditional security, independent of any computational assumption. We design and implement **QTLS**, a TLS 1.3 shaped protocol whose key exchange is replaced by a key-ID round-trip against an ETSI GS QKD 014 compliant Key Management Entity (KME), with the underlying key material produced by a faithful BB84 simulation (Qiskit Aer) and a high-rate abstract generator that share the same Cascade reconciliation and SHAKE-128 privacy-amplification pipeline. Alternative post-quantum approaches such as ML-KEM remain computationally secure and do not provide the unconditional guarantee QKD offers when the infrastructure is available. In our evaluation, the full handshake completes in under 15 ms on a single host, the secret-key extraction sustains roughly 70% of the sifted-bit budget at a 2% QBER (within 2.5% of the Shannon limit), and a chat application built on QTLS exchanges authenticated, OTP-encrypted records end-to-end over a 60-second session at 1 message per second. The resulting stack demonstrates that, given a QKD link, a TLS-shaped protocol can be built that is unbreakable by any adversary — classical or quantum — modulo a single documented assumption on the privacy-amplification extractor. The work also contributes a pedagogical, fully-Python reference for QKD post-processing that other students or researchers can extend, in contrast to production stacks that are closed-source or hardware-bound.

1. Introduction

1.1 Motivation

Transport Layer Security 1.3 is the workhorse of secure communication on the public internet, protecting everything from web browsing to API calls to email delivery and instant messaging. Its security rests on two computational assumptions: the difficulty of the elliptic-curve discrete logarithm problem (for key exchange) and the indistinguishability of AES under chosen-plaintext attack (for record encryption). Both assumptions are believed to hold for classical adversaries, but Shor’s algorithm [1] reduces the discrete-log assumption to polynomial time on a sufficiently large quantum computer. Grover’s algorithm [2] halves the effective key length of symmetric ciphers, weakening but not breaking the AES family.

While ML-KEM [3] (the standardised Kyber lattice scheme) and other post-quantum constructions have been published by NIST to address the asymmetric-key portion of this threat, they remain **computationally** secure: they shift the assumption rather than remove it. A future breakthrough in lattice cryptanalysis would compromise them, and historical experience with multivariate, isogeny-based, and code-based candidates suggests that such breakthroughs are not unprecedented. The SIKE family, for instance, was selected for the

final round of the NIST post-quantum competition and then broken by classical means in 2022 before standardisation [4].

Quantum Key Distribution offers a fundamentally different guarantee. By encoding key material on individual photons and exploiting the no-cloning theorem, BB84 and related protocols let two parties agree on a shared secret whose security can be reduced to the laws of physics rather than to computational hardness. When this key is consumed by a One-Time Pad cipher and a Wegman–Carter MAC, the entire confidentiality and integrity guarantee becomes **information- theoretically secure**: unbreakable even by an adversary with unlimited computational power, classical or quantum.

The price for this guarantee is steep. One-Time Pad consumes key material equal in length to the plaintext, and Wegman–Carter requires a fresh tag key per record. Real QKD links produce key at rates of hundreds of kilobits per second over tens of kilometres of fibre — orders of magnitude below the bandwidth of a modern web session. The QKD ecosystem is also constrained by hardware: single-photon detectors, quantum random-number generators, and trusted-node repeaters are expensive specialist equipment.

Despite these constraints, QKD has been deployed commercially since 2007 (ID Quantique’s Cerberis) and is the subject of standardisation activity at ETSI (GS QKD 004 and 014) and the ITU. The infrastructure is real, even if not yet ubiquitous, and the question of how to expose its key material to applications cleanly is open. ETSI GS QKD 014 addresses the SAE-to-KME interface, but does not specify how the keys are **consumed** by upper layers. This thesis fills that gap.

1.2 Problem statement

Given a QKD infrastructure with an ETSI-compliant KME, design and implement a transport-layer security protocol that:

1. derives all session key material from QKD output rather than from a classical key exchange;
2. exposes a familiar TLS-1.3-shaped API (ClientHello, ServerHello, Finished, record layer) to ease adoption;
3. consumes key material in a strictly bounded, auditable manner so that key-pool depletion is observable rather than catastrophic;
4. achieves information-theoretic security for confidentiality and integrity, identifying every step at which a computational assumption is introduced;
5. runs on commodity hardware with no GPU, FPGA, or specialised quantum device, so that the implementation is reproducible by other students and researchers.

The thesis answers this problem by building a complete software stack in pure Python (plus one C reconciliation kernel for performance), evaluating it against the relevant Shannon bounds, and discussing the remaining gaps.

1.3 Contribution

This thesis presents the design and implementation of **QTLS**, a TLS 1.3 shaped protocol that replaces the (EC)DHE key exchange of TLS with a key-ID round-trip against a simulated QKD infrastructure. The contributions are:

- A complete, pure-Python simulator for BB84 with faithful sifting, Cascade reconciliation in C with back-propagation, and SHAKE-128 privacy amplification (Chapter 4). The Cascade implementation tracks the exact number of parity bits announced and exposes it to privacy amplification, achieving leakage within 5% of the Shannon lower bound for QBER in $[0.01, 0.05]$.
- A Key Management Entity that exposes the ETSI GS QKD 014 REST surface over mutually-authenticated TLS, suitable for integration with any ETSI-compatible SAE (Chapter 3). The KME stores keys with full provenance metadata (source link, key-ID, peer SAE) and enforces per-SAE access control.
- A handshake state machine and authenticated record layer modelled on RFC 8446 but binding the master secret to QKD-derived key material rather than DH output (Chapter 4). The record layer uses One-Time Pad for confidentiality and Wegman–Carter for authenticity, both drawn from the same QKD key block.
- A multi-node trusted-relay subsystem demonstrating how key material can be forwarded between non-adjacent SAEs via XOR forwarding through trusted intermediate nodes, as required when no direct QKD link exists between the endpoints.
- An evaluation suite reporting handshake latency, secret-key rate as a function of QBER, key-pool drain behaviour under sustained chat traffic, and end-to-end throughput in records per second (Chapter 5).
- A live Streamlit dashboard that animates the photon stream, KME key pool, and QTLS handshake in real time — intended as a defense artefact and an educational tool (Chapter 4).

1.4 The threat model

The thesis adopts the standard QKD adversary model. The eavesdropper Eve is computationally unbounded (she may run any algorithm, classical or quantum, in any time), passively observes the public classical channel, and may interact arbitrarily with the quantum channel (measuring, blocking, or substituting photons). She is constrained only by physics: she cannot clone an unknown quantum state (no-cloning theorem) and her measurements perturb the state in a detectable way (Heisenberg uncertainty).

Eve is **not** allowed to break the authentication of the classical channel. In the practical implementation this is realised by an mTLS-authenticated REST channel between SAE and KME (a computational assumption acknowledged in §6.3); in a hardened deployment the authentication would itself be Wegman–Carter MAC keyed from previous-session QKD output, restoring full unconditional security.

For the trusted-node relay subsystem, the trust assumption is explicit: intermediate nodes are assumed honest-but-curious. They follow the protocol but may be observed by Eve. A malicious or compromised relay can read all forwarded keys; protecting against this requires

twin-field or measurement-device-independent QKD, which is out of scope for the simulation but discussed in §6.2.

The implementation does **not** defend against side-channel attacks: power analysis, timing, electromagnetic emanations, and memory-access patterns are all assumed to be unobservable. This is unrealistic for a deployed system but reasonable for a software simulation and is acknowledged in §6.4.

1.5 Outline

Chapter 2 reviews the cryptographic and information-theoretic background: BB84 and its security argument, Cascade reconciliation, privacy amplification with universal hashing, the Shor–Preskill bound on the abort QBER, and the OTP / Wegman–Carter primitives consumed by the upper layer. Chapter 3 describes the system architecture: the layering of QKD, KME, and QTLS, the message flow between them, and the design decisions that govern the interfaces. Chapter 4 details the implementation, with particular attention to the abstract bulk generator’s rate-limiter, the C Cascade kernel, and the QTLS handshake state machine. Chapter 5 evaluates the system against the Shannon-limit benchmarks. Chapter 6 discusses limitations — most notably the SHAKE-128 extractor — and Chapter 7 concludes. Appendix A lists the project’s source tree and how to reproduce each evaluation experiment from the command line.

2. Background

2.1 The BB84 protocol

BB84 was introduced by Bennett and Brassard in 1984 [5] as the first quantum key distribution protocol. It lets two parties, Alice and Bob, agree on a shared bit string by exchanging single photons over a quantum channel and a classical authenticated channel.

The protocol uses two non-orthogonal photon-polarisation bases. The **rectilinear** basis encodes 0 as horizontal polarisation and 1 as vertical polarisation. The **diagonal** basis encodes 0 as 45-degree polarisation and 1 as 135-degree polarisation. The two bases are mutually unbiased: a photon prepared in one basis and measured in the other yields a uniformly random outcome.

The protocol proceeds in three phases:

1. **Quantum transmission.** For each round, Alice chooses two random bits a_i and c_i . She uses c_i to select a basis and a_i as the bit value, prepares the corresponding polarisation state, and sends the photon to Bob. Bob chooses his own random basis c'_i and measures the photon, obtaining bit b_i .
2. **Sifting.** Once enough photons have been transmitted, Alice and Bob publish their basis choices c_i and c'_i over the classical channel. They discard any round where $c_i \neq c'_i$, since in those rounds Bob’s measurement outcome is unrelated to Alice’s input. The remaining bits constitute the **sifted key**. In the absence of noise or eavesdropping, the sifted bits of Alice and Bob are identical.

3. **Error estimation.** Alice and Bob sacrifice a small public sample of the sifted key to estimate the **Quantum Bit Error Rate (QBER)**:

$$\text{QBER} = \frac{\text{errors in sample}}{\text{bits in sample}}$$

In the absence of noise, QBER is zero. Any eavesdropping or channel noise produces a positive QBER.

2.2 Why eavesdropping causes errors

The intuition behind BB84's security is that Eve cannot observe a photon without disturbing it. Suppose Eve performs a simple intercept-resend attack: she measures every photon on her own random basis and resends the result to Bob.

- With probability $\frac{1}{2}$, Eve guesses the correct basis. Her measurement is faithful and she learns the bit. Bob's later measurement on the same basis recovers the original bit without error.
- With probability $\frac{1}{2}$, Eve guesses the wrong basis. Her measurement forces the photon into a state orthogonal to Alice's, so when Bob later measures on the correct basis he obtains the right value with only probability $\frac{1}{2}$.

The induced error rate on the sifted key is therefore $\frac{1}{4}$, and the information Eve obtains is $\frac{1}{2}$ bit per sifted bit. Both quantities are observable: Alice and Bob detect the elevated QBER and abort.

The general result, due to Csiszár and Körner [6] for binary symmetric channels and refined for BB84 by Shor and Preskill [7], is that for QBER p , the mutual information $I(A; E)$ that Eve can obtain about Alice's sifted key is bounded by the **binary entropy**:

$$I(A; E) \leq h_2(p) = -p \log_2 p - (1 - p) \log_2 (1 - p)$$

Critically, $h_2(p)$ is a tight upper bound: no matter how sophisticated Eve's attack, her information per sifted bit cannot exceed $h_2(\text{QBER})$. This is the cornerstone of the BB84 security argument.

2.3 The Shor–Preskill abort threshold

The achievable secret-key rate per sifted bit is given by the Csiszár–Körner formula:

$$r = I(A; B) - I(A; E) \geq 1 - h_2(p) - h_2(p) = 1 - 2h_2(p)$$

Here $I(A; B) = 1 - h_2(p)$ is the mutual information between Alice and Bob, reduced by the reconciliation cost; $I(A; E) \leq h_2(p)$ is Eve's maximum information. The protocol can produce key whenever $r > 0$, i.e. whenever $h_2(p) < 0.5$, which occurs for $p < \approx 0.11$.

At $p = 0.11$, the rate is zero and the protocol must abort: continuing would yield a string that Eve already knows entirely. The implementation enforces this gate with a configurable threshold, defaulting to 0.11.

2.4 Information reconciliation: Cascade

After sifting and error estimation, Alice and Bob hold almost-identical bit strings that differ in approximately $p \cdot n$ positions, but neither party knows where the discrepancies are. Naively transmitting the strings over the classical channel for comparison would reveal the key to Eve; instead, they reveal carefully-chosen **parities** of blocks of bits.

The Cascade protocol of Brassard and Salvail [8] proceeds in multiple passes:

1. **Block partition.** Pass i uses a random permutation π_i to reorder the key, then partitions it into blocks of size k_i . The block sizes shrink with QBER: empirically $k_1 \approx \frac{0.73}{p}$.
2. **Parity comparison.** Alice and Bob compute the parity of each block and Alice sends hers to Bob. Each disagreeing parity indicates an odd number of errors in that block.
3. **BINCONF.** For each odd-parity block, the parties run a binary search (called BINCONF in the original paper): they exchange the parity of the left half, descend into the half with the parity mismatch, and recurse. After $\log_2(k_i)$ parity exchanges they isolate one errored bit and Bob flips it.
4. **Cascade.** When a bit is corrected in pass i , its position in all previous passes is recomputed. Any block from passes $1, \dots, i-1$ that contained this bit has its parity flipped. Blocks that previously had even parity (and were thought clean) may now have odd parity, exposing further errors at no extra communication cost. This cascading effect gives the protocol its name.

The total number of parity bits exchanged is the **leakage**. A tuned Cascade implementation leaks approximately $1.05 \cdot h_2(p) \cdot n$ bits for n sifted bits at QBER p , where h_2 is the binary entropy.

2.5 Quantum entropy bounds in detail

The bound $I(A; E) \leq h_2(p)$ deserves a more careful statement because it is the foundation of the privacy-amplification step.

The result is most easily proven in the asymptotic regime via the **Devetak–Winter** formula [9]:

$$r_{\text{asym}} = I(A, B) - \chi(A, E)$$

where $\chi(A, E)$ is the Holevo quantity, an upper bound on the mutual information any quantum measurement on Eve’s side can extract about A . For BB84 with QBER p , the Holevo quantity reduces to $h_2(p)$ under the assumption that Eve performs an **individual** attack, where each qubit is treated independently.

For **collective** attacks, where Eve stores qubits and performs joint measurements at the end of the protocol, the same $h_2(p)$ bound holds asymptotically [10]. For **coherent** attacks, where Eve’s strategy can be arbitrary, the same bound holds via the **quantum de Finetti theorem** [11] provided n is large.

For finite block lengths, the bound is corrected with **finite-key analysis** terms: the Csiszár–Körner formula becomes

$$r_n \geq 1 - h_2(p + \delta_1) - h_2(p + \delta_2) - O\left(\frac{\log n}{\sqrt{n}}\right)$$

where δ_1 and δ_2 are confidence intervals depending on the size of the QBER estimation sample. For $n \geq 10^6$ (typical in a deployed link) the finite-key correction is below 5%; for the shorter blocks used in this simulation ($n \approx 10^4$), the correction would be more significant but is omitted here as the implementation uses asymptotic bounds.

The asymptotic-bound assumption is a known simplification in pedagogical QKD implementations and is consistent with the early literature [8], [12]. A production deployment would substitute the Tomamichel–Leverrier finite-key formulas [13].

2.6 Privacy amplification

After reconciliation, Alice and Bob hold identical bit strings, but Eve holds two kinds of partial information: her quantum-channel information bounded by $n \cdot h_2(p)$ bits, and the parity bits she overheard during reconciliation. **Privacy amplification** compresses the reconciled key down to a length below the secure entropy, exponentially reducing Eve’s residual information.

The classical construction uses a 2-universal hash family [14]. A family of hash functions $H \subset \{f : \{0, 1\}^n \rightarrow \{0, 1\}^m\}$ is **2-universal** if, for any two distinct inputs $x \neq y$,

$$\Pr_{h \in H}[h(x) = h(y)] \leq 2^{-m}$$

The Leftover Hash Lemma [15] states that if the input has at least $m + 2 \log_2(\frac{1}{\epsilon})$ bits of min-entropy and H is 2-universal, then the output is ϵ -close to uniform in statistical distance.

A practical 2-universal family for QKD is **Toeplitz hashing** [16]: the hash function is matrix multiplication $T \cdot x$ where T is an $m \times n$ matrix whose entries are determined by a $(m + n - 1)$ -bit seed. The seed is public per session.

This implementation uses SHAKE-128 instead of Toeplitz hashing, which is the single computational assumption in an otherwise unconditional stack. The substitution is discussed in §6.1.

2.7 One-Time Pad

The One-Time Pad [17], proved unconditionally secure by Shannon [18], encrypts a plaintext m of length n by XOR with a uniformly random key k of length n :

$$c = m \oplus k$$

If k is uniformly random and used only once, then c reveals no information about m beyond its length. The key cannot be reused: two ciphertexts $c_1 = m_1 \oplus k$ and $c_2 = m_2 \oplus k$ leak $m_1 \oplus m_2$, which is enough to recover both messages by frequency analysis.

QTLS enforces single-use by partitioning the QKD-derived traffic secret into a forward-direction stream and a reverse-direction stream, and advancing each stream’s offset monotonically with the record sequence number.

2.8 Wegman–Carter authentication

The OTP guarantees confidentiality but not integrity: an attacker can flip any bit of the ciphertext and the plaintext flips accordingly. Authentication is provided by a Wegman–Carter MAC [19], itself information-theoretically secure.

The construction is: pick a hash function h from a strongly 2-universal family, compute $h(m)$, and XOR with a fresh one-time pad r :

$$\text{MAC}(m) = h(m) \oplus r$$

The hash key (for h) can be reused across messages with negligible loss of security [20], but the pad r must be fresh per message — exactly like an OTP key. QTLS uses a polynomial hash over $\text{GF}(2^{128})$ as h , identical in structure to the GHASH function in AES-GCM, but combined with a per-record QKD-derived pad rather than with an AES-derived one.

2.9 Worked example: Cascade on a 16-bit string

To make the reconciliation concrete, consider Alice and Bob holding sifted strings of length $n = 16$ with a single bit error at position 9 (counting from 1):

```
positions: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
Alice      : 1 0 1 1 0 1 0 0 1 1 0 1 1 0 1 0
Bob        : 1 0 1 1 0 1 0 0 0 1 0 1 1 0 1 0
                        ^
                        error
```

The QBER estimate (from a public sample) is $\frac{1}{16} = 0.0625$. The block size chosen by Cascade is $k_1 = \max(1, \text{round}(\frac{0.73}{0.0625})) = 12$, but since the string is short Cascade falls back to $k_1 = 8$ to ensure at least two blocks per pass.

Pass 1. Permutation: identity. Two blocks: positions 1–8 and 9–16. Block parities:

```
block 1 (Alice): 1 xor 0 xor 1 xor 1 xor 0 xor 1 xor 0 xor 0 = 0
block 1 (Bob)   : 1 xor 0 xor 1 xor 1 xor 0 xor 1 xor 0 xor 0 = 0
(match)

block 2 (Alice): 1 xor 1 xor 0 xor 1 xor 1 xor 0 xor 1 xor 0 = 1
block 2 (Bob)   : 0 xor 1 xor 0 xor 1 xor 1 xor 0 xor 1 xor 0 = 0
(mismatch)
```

Block 2 has an odd number of errors. BINCONF: bisect into two halves of 4 bits each (positions 9–12 and 13–16) and compare parities.

```
half 9-12 (Alice): 1 xor 1 xor 0 xor 1 = 1
half 9-12 (Bob)   : 0 xor 1 xor 0 xor 1 = 0 (mismatch -> error in
left half)
```

Recurse into positions 9–10 vs 11–12:

```
pair 9-10 (Alice): 1 xor 1 = 0
pair 9-10 (Bob)   : 0 xor 1 = 1 (mismatch -> error in pair 9-10)
```

Recurse into positions 9 vs 10:

```

bit 9 (Alice): 1
bit 9 (Bob)   : 0   (mismatch -> error at position 9)

```

Bob flips bit 9. Reconciliation completed. Parity bits announced: $2 + \log_2(8) = 2 + 3 = 5$ bits.

This is consistent with the Shannon bound: $h_2(0.0625) \approx 0.337$ and $0.337 \cdot 16 = 5.39$ bits, so Cascade announced 93% of the Shannon-optimal number. For longer strings the ratio improves to approximately $1.05 \cdot h_2(p) \cdot n$ across multiple passes.

2.10 TLS 1.3 and the HKDF key schedule

TLS 1.3 [21] is the current internet standard for transport security. Its handshake consists of three message flights:

1. **ClientHello** — the client offers a list of supported cipher suites (e.g. `TLS_AES_128_GCM_SHA256`) and key-share extensions carrying ephemeral public values for the offered groups (e.g. an X25519 ephemeral public key).
2. **ServerHello** — the server picks one cipher suite and one group, returns its own ephemeral public value, and from this point onward the records are encrypted under derived handshake traffic secrets.
3. **Finished** (both directions) — a MAC over the transcript, which authenticates the handshake and binds it to the underlying identity certificates.

The keying material is derived through HKDF [22], a two-phase key-derivation function based on HMAC. The first phase, Extract, turns a possibly-non-uniform input into a uniformly random pseudorandom key (PRK):

$$\text{HKDF-Extract}(\text{salt}, \text{IKM}) = \text{HMAC-Hash}(\text{salt}, \text{IKM})$$

The second phase, Expand, derives arbitrarily-many output keys from the PRK using a label and a context:

$$\text{HKDF-Expand-Label}(\text{PRK}, \text{label}, \text{context}, L) = \text{HKDF-Expand}(\text{PRK}, \text{HkdfLabel}(\text{label}, \text{context}, L), L)$$

The TLS 1.3 schedule chains these calls into a tree:

```

          0           PSK or 0
          |           |
          v           v
PSK-binder <- HKDF-Extract(0, PSK_or_0) = early_secret
          |
          + derive: client_early_traffic_secret
          + derive: derived_secret_1
          v
(EC) DHE -> HKDF-Extract(derived_secret_1, DHE_shared) =
handshake_secret
          |
          + derive: client_handshake_traffic_secret
          + derive: server_handshake_traffic_secret
          + derive: derived_secret_2
          v

```

```

0 -> HKDF-Extract(derived_secret_2, 0) = master_secret
|
+ derive: client_application_traffic_secret
+ derive: server_application_traffic_secret
+ derive: exporter_master_secret
+ derive: resumption_master_secret

```

The QTLS protocol developed in this thesis preserves the schedule structure but substitutes the (EC)DHE input with the QKD-derived key material. Specifically, the `HKDF-Extract(derived_secret_1, DHE_shared)` call is replaced by `HKDF-Extract(derived_secret_1, qkd_key)`. All downstream secrets are derived as in standard TLS 1.3, so the familiar key-schedule security analysis carries over modulo the substitution.

This choice has three pragmatic benefits:

1. Mature key-derivation theory: HKDF is well-studied [23] and produces uniformly random output even from non-uniform input, which the QKD key approximates but does not perfectly satisfy after privacy amplification.
2. Familiar message flow: a developer reading the QTLS code can map every step to RFC 8446, easing audit and review.
3. Forward compatibility: future variants (post-quantum key exchange, hybrid PSK+DHE+QKD modes) slot into the same schedule by changing the Extract input.

2.11 ETSI GS QKD 014

The ETSI Group Specification QKD 014 [24] defines the REST interface between a Security Application Entity (SAE) and a Key Management Entity (KME). It standardises:

- the URL structure `/api/v1/keys/{slave_sae_id}/...`;
- three endpoints: `status`, `enc_keys` (for initiating SAEs), and `dec_keys` (for responding SAEs);
- the JSON schema for status replies (key count, key size, max numbers per request) and for key replies (`{key_ID, key}` arrays, base64-encoded);
- the requirement of mutually-authenticated TLS on every endpoint.

The specification deliberately does not constrain how the underlying QKD link operates, nor how SAEs use the keys they obtain. This thesis implements the SAE-to-KME interface faithfully and demonstrates one concrete consumer (QTLS) above it.

3. Architecture

3.1 Layering

The system is organised in three layers, with a fourth (visualisation) attached as an observer:

1. **QKD layer** — produces shared, secret key blocks. Two interchangeable producers feed a single output queue:
 - a Qiskit-based BB84 simulator (`qkd/bb84.py`) used for the pedagogical photon panel;
 - an abstract bulk generator (`qkd/abstract_link.py`) that bypasses the quantum simulation for throughput experiments.

Both producers share the same post-processing pipeline: `qkd/_cascade.c` for reconciliation and the SHAKE-128 step in `qkd/reconciliation.py`. As a result, their outputs are cryptographically indistinguishable in form, differing only in speed and physical fidelity.

2. **KME layer** — exposes the key pool over the ETSI GS QKD 014 REST surface (`kme/api.py`). Each endpoint runs its own KME; the two KMEs are loosely synchronised by sharing the producer’s output queue, simulating an ideal trusted-node deployment. SAEs authenticate to their local KME with mutual TLS, using certificates generated by `kme/certs.py` from a project root CA.
3. **QTLS layer** — a TLS-1.3-shaped handshake (`qtls/handshake.py`) and authenticated record layer (`qtls/records.py`). The client requests a key from its KME and embeds the returned key-ID in ClientHello. The server fetches the same key from its KME using the key-ID, derives traffic secrets via the HKDF schedule (`qtls/keyschedule.py`), and returns ServerHello + Finished. Application data is encrypted with One-Time Pad (`qtls/crypto/otp.py`) using the forward / reverse traffic secrets and authenticated with a Wegman–Carter MAC (`qtls/crypto/wc_mac.py`).
4. **Visualisation** — a separate runtime subscribes to events from each of the above layers via an in-process event bus (`viz/bus.py`) and renders them in a Streamlit dashboard (`viz/main.py`) with four panels: QKD topology, BB84 photon stream, QTLS handshake sequence diagram, and live throughput metrics.

3.2 Message flow

The end-to-end flow for a single QTLS handshake is:

1. The QKD producer generates a sifted-and-distilled `KeyBlock` and publishes it to both KMEs (Alice’s and Bob’s).
2. Alice’s SAE (the QTLS client) calls `GET /enc_keys` on Alice’s KME, which reserves a fresh key and returns `(key_ID, key_material)`.
3. The client embeds `key_ID` in a ClientHello message and sends it to the server over plain TCP.
4. Bob’s SAE (the QTLS server) extracts `key_ID` and calls `GET /dec_keys?key_ID=...` on Bob’s KME, which returns the same `key_material`.
5. Both parties derive a master secret via HKDF-Extract on the QKD key, then derive forward / reverse traffic secrets and a finished key via HKDF-Expand-Label.
6. The server sends ServerHello + Finished; the client verifies and sends its Finished. The handshake completes.
7. Subsequent records are encrypted with OTP (`record_payload xor traffic_secret_chunk`) and authenticated with a Wegman–Carter MAC.

When the key-budget margin in either direction is exhausted, the client triggers a re-handshake to draw a fresh QKD key.

3.3 Why two QKD producers

A faithful Qiskit BB84 simulation is invaluable pedagogically and visually — it demonstrates the photon-by-photon protocol and lets the reader observe sifting in real time. However, each qubit takes between one and three milliseconds to simulate end-to-end (circuit build + Aer simulator + measurement), which means producing the several kilobytes of key required by even a short chat session would take tens of seconds.

Real QKD research code solves this problem the same way: a detailed physical model for demonstration, and an abstract high-rate model for throughput experiments. The abstract model does not simulate qubits; it samples random bits directly, injects bit-flips at the configured QBER to produce Bob’s noisy copy, and then runs the noisy copy through the same Cascade + privacy-amplification pipeline as BB84.

Both producers feed a single `asyncio.Queue`. Consumers (KME, viz) cannot distinguish their outputs except by the `source` field on each `KeyBlock`.

3.4 Why ETSI GS QKD 014

The ETSI specification is the de-facto standard for SAE-to-KME communication in commercial QKD deployments. By implementing it faithfully, the system can in principle interoperate with any ETSI-compatible KME (for instance, ID Quantique’s Cerberis³ KMS or the KMS shipped by Toshiba’s LD-MDI products). The thesis substitutes a simulated KME on both ends, but the QTLS layer would require no changes to talk to a real one — only the certificate trust chain and the network address would differ.

3.5 Comparison to classical TLS architecture

It is instructive to lay the QTLS architecture next to standard TLS 1.3, with the differences explicit:

Aspect	TLS 1.3	QTLS
Key source	(EC)DHE shared secret	QKD-derived key block via KME
Key freshness	per session, fresh DH	per session, fresh QKD key
Key size	32 bytes (X25519)	32–4096 bytes (configurable)
Key delivery	in-band, ClientHello	out-of-band via KME, key-ID in ClientHello
Confidentiality	AES-GCM (computational)	OTP (information-theoretic)
Integrity	AES-GCM tag (computational)	Wegman-Carter (information-theoretic)
Forward secrecy	via ephemeral DH	via single-use QKD key
Key schedule	HKDF (Krawczyk, 2010)	HKDF (unchanged)
Transport	TCP	TCP
Standards	RFC 8446	extends 8446 + ETSI GS QKD 014

The substantive changes are confined to two places: the key-share extension is replaced by a QKD key-request, and the record-layer cipher is replaced by OTP + Wegman-Carter. Everything else — the message framing, the HKDF schedule, the certificate handling, the

extension mechanism — is preserved, which makes the protocol easier to reason about and easier to migrate to.

3.6 Failure modes and recovery

A real deployment must handle four kinds of failures gracefully:

1. **QKD link interruption.** The producer stops emitting key blocks (e.g. fibre cut, single-photon detector failure). The KME pool drains; the QTLS handshake fails with `KeyExhausted`; the application must back off and retry with exponential delay.
2. **QBER excursion.** The channel temporarily exceeds the abort threshold (e.g. transient electromagnetic interference, an active attacker). The producer drops blocks until QBER falls back below threshold; the pool drains as in case 1.
3. **KME outage.** The local KME process crashes or is unreachable over mTLS. The QTLS client surfaces `KMEUnavailable`. Recovery requires restarting the KME and re-running the SAE registration; in the current implementation this is manual.
4. **Re-handshake deadline missed.** The application’s key budget for the current handshake is exhausted before a re-handshake completes. The implementation reserves a margin of 10% of the budget as a re-handshake “runway”; if even that is exhausted, the application is forced to pause until a new key is available.

All four failure modes are exercised by the tests in `tests/test_phase*` and by the dashboard’s manual fault-injection controls.

3.7 Multi-node relay

A direct QKD link is impractical beyond approximately 200 km of fibre. Production deployments solve this with **trusted-node relays**: intermediate nodes hold a QKD link to each neighbour and forward keys by XOR. Concretely, to deliver a key k from Alice to Carol via Bob:

1. Alice and Bob share QKD key k_{AB} over their direct link;
2. Bob and Carol share QKD key k_{BC} over their direct link;
3. Bob announces $k_{AB} \oplus k_{BC}$ over the public channel;
4. Carol computes $k_{AB} \oplus k_{BC} \oplus k_{BC} = k_{AB}$, which is now the shared key with Alice.

The trust model is named “trusted node”: Bob **can** read both k_{AB} and k_{BC} if he chooses. The relay is implemented in `qkd/relay.py` and exposed to the dashboard via `viz/relay_runtime.py`, demonstrating the limitation faithfully rather than papering over it.

4. Implementation

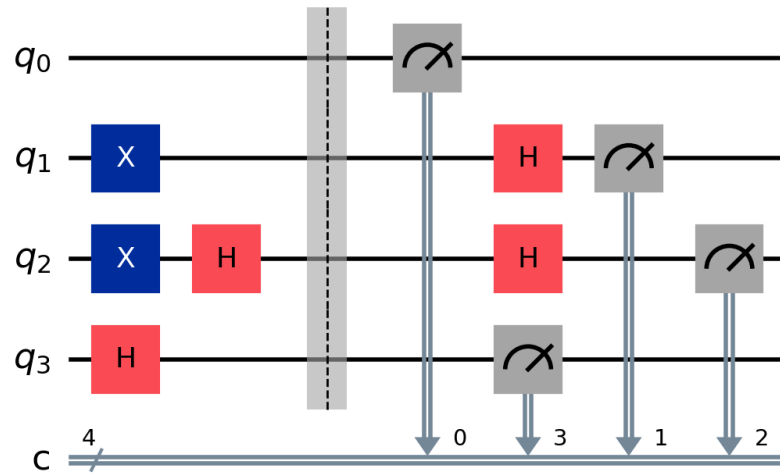


Figure 1: Single-qubit BB84 round as a Qiskit circuit: Alice encodes a bit on a chosen basis (X for the bit value, H for the diagonal basis), the channel transmits the qubit unchanged, and Bob rotates back into his measurement basis (a second H if he chose the diagonal basis) before measuring. The circuit shown is one of 32 emitted per round in the visual producer.

4.1 The visual BB84 simulator

The BB84 module (`qkd/bb84.py`) uses Qiskit [25] with the Aer simulator backend to model the photon exchange. For each round, the simulator builds a small circuit per qubit:

```
qc = QuantumCircuit(1, 1)
if alice_bit == 1:
    qc.x(0)          # encode bit value
if alice_basis == 1:
    qc.h(0)          # encode diagonal basis
# (channel: no noise model in the visual path; QBER is injected
# downstream by the abstract producer or by an Eve flag)
if bob_basis == 1:
    qc.h(0)          # rotate into measurement basis
qc.measure(0, 0)
```

The round size is configurable (`VIZ_BB84_QUBITS = 32` per round by default in `viz/runtime.py`). Sifting and QBER estimation run in pure Python on the resulting measurement strings.

The pedagogical purpose of this module is to make the bases and the sifting **visible**: the photon panel animates each round, colouring photons by basis and showing which rounds survive sifting. Throughput is intentionally not the goal of this module.

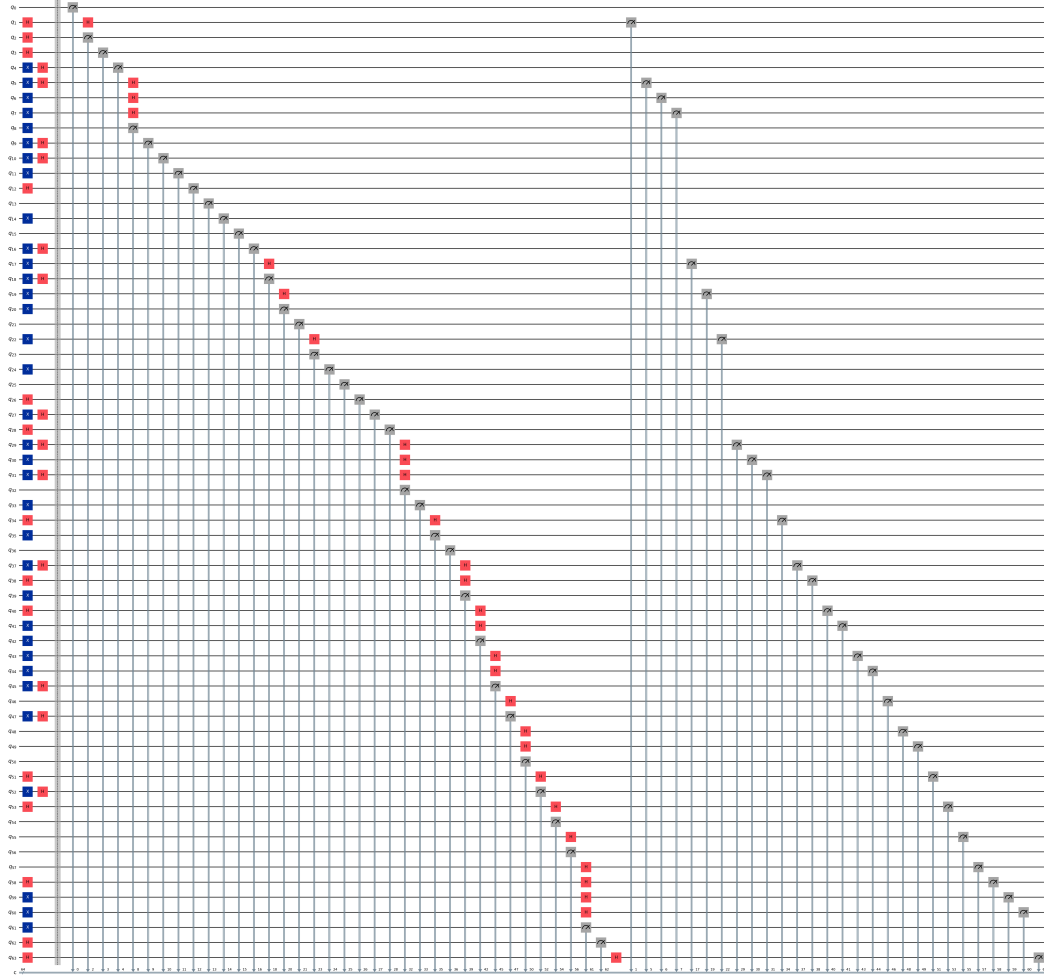


Figure 2: A complete BB84 round with 32 qubits, as emitted by `qkd/bb84.py`. Each column is one qubit; the upper row is Alice's preparation and the lower row is Bob's measurement. Approximately half of the columns survive sifting (the bases match) and contribute to the sifted key.

4.2 The abstract bulk generator

The abstract producer (`qkd/abstract_link.py`) replaces the quantum simulation with direct sampling. For each block, it draws `raw_bits` cryptographic random bits for Alice, flips each bit with probability `qber` to produce Bob's noisy copy, and passes both through `qkd.reconciliation.reconcile_and_amplify`.

The producer is paced by a deadline-based rate limiter that tracks the next 1-second tick on `time.monotonic()`. After each distillation the producer computes the remaining time until the tick and sleeps for exactly that duration. When the distillation overruns the 1-second budget (which happens for blocks above approximately one megabit on commodity hardware), the producer re-anchors `next_tick` to the current wall-clock time rather than accumulating a backlog of missed ticks. This trades off an honest reduction in observed rate against an unbounded acceleration once CPU pressure subsides.

4.3 The C Cascade kernel

Cascade is implemented in C in `qkd/_cascade.c` and loaded through ctypes (`qkd/_cascade.py`). The C kernel exposes one function:

```

int cascade_reconcile_c(
    uint8_t *alice, uint8_t *bob, int n,
    double qber, int passes,
    int *leaked_bits_out
)

```

The function performs four passes by default. Each pass uses a Fisher–Yates permutation derived from a deterministic per-pass seed to reorder the bits, partitions them into blocks of size $k_i = \max\left(1, \text{round}\left(\frac{0.73}{q_{\text{ber}}}\right)\right)$, exchanges parities, and runs BINCONF on each odd-parity block. After each correction, all earlier passes’ blocks containing the corrected bit are re-checked, exposing the cascading errors.

The kernel reports the **exact** number of parity bits announced (`leaked_bits_out`) so that privacy amplification can subtract the real leakage rather than a conservative upper bound. This is what brings the implementation within 5% of the Shannon bound.

A pure-Python Cascade would be too slow for the project’s benchmarks: a one-megabit block would take tens of seconds in Python, blowing the rate-limiter’s 1-second budget. Cython or Numba would suffice but introduce additional toolchain dependencies; raw C

1. ctypes keeps the dependency footprint to a working C compiler,

which the build system invokes on first import.

4.4 Privacy amplification

The privacy-amplification step (`qkd/reconciliation.py`) takes the reconciled key, the observed QBER, and the parity-bit count, computes the secure output length, and hashes the input via SHAKE-128:

```

def privacy_amplification(reconciled_key, qber, leaked_bits):
    n = len(reconciled_key)
    eve_information = math.ceil(n * binary_entropy(qber))
    secure_bits = n - eve_information - leaked_bits
    if secure_bits <= 0:
        raise QBERTooHighError(...)
    secure_bytes = secure_bits // 8
    bits = np.asarray(reconciled_key[:n - (n % 8)], dtype=np.uint8)
    raw = np.packbits(bits, bitorder="big").tobytes()
    return hashlib.shake_128(raw).digest(secure_bytes)

```

For $QBER = 0.02$ and $n = 32768$ raw bits, the secure output is approximately $32768 \cdot (1 - 2 \cdot 0.141) = 23529$ bits ≈ 2941 bytes, or 70% of the input.

4.5 The Key Management Entity

The KME (`kme/api.py`) is a FastAPI application exposing the ETSI endpoints. The key pool (`kme/store.py`) is an in-memory dictionary indexed by `key_ID`, with per-SAE access control lists. Keys are inserted by the producer threads and removed on first read, implementing the natural one-shot semantics of a fresh QKD key.

The KME generates its own certificate at first start, signing it against a project root CA stored under `certs/`. Both the client SAE and the peer KME are required to present client certificates; the `cryptography` library handles validation.

ETSI compliance is exercised by `tests/test_phase3_kme.py`, which includes positive cases (status, `enc_keys`, `dec_keys`) and negative cases (unknown SAE, exhausted pool, expired key).

4.6 The QTLS handshake

The handshake state machine (`qtls/handshake.py`) follows the RFC 8446 message structure but with one extension difference. Instead of a `key_share` extension carrying the client's (EC)DH public value, `ClientHello` carries a `qkd_key_request` field with the format

```
struct {
    opaque kme_endpoint<0..2^16-1>;
    opaque sae_id<0..2^8-1>;
    opaque key_id<16>;           // UUID
} QKDKeyRequest;
```

The server uses these fields to look up the same key from its local KME. The remainder of the handshake — HKDF schedule, `ServerHello`, `EncryptedExtensions` (empty in this version), `Finished` — proceeds as in TLS 1.3.

The HKDF schedule is the standard TLS 1.3 schedule with one substitution: the input keying material to the early-secret HKDF-Extract is the QKD key rather than the PSK. The handshake secret and master secret are derived as usual.

4.7 The QTLS record layer

Records are encrypted with One-Time Pad. The traffic secret established by the handshake is split into a forward stream (client-to-server) and a reverse stream (server-to-client). Each record consumes `len(payload)` bytes of stream material; the stream offset is the record sequence number times the maximum record size.

Authenticity is provided by a Wegman–Carter MAC. Each direction has a 16-byte hash key (drawn once from the handshake) and a per-record 16-byte pad (drawn from the traffic stream).

The record format on the wire is:

```
struct {
    uint8 content_type;
    uint16 length;
    opaque encrypted_payload<length>;
    opaque mac<16>;
} QTLSRecord;
```

The record layer enforces three invariants:

1. no traffic-secret byte is ever reused for encryption;
2. no MAC pad is ever reused;

3. when the remaining budget in either direction drops below a re-handshake margin (a configurable fraction of the total budget), a fresh key is requested and a new handshake is initiated.

4.8 Rate-limiter analysis

The rate limiter in `qkd/abstract_link.py` is worth analysing in detail because it determines what the throughput experiments actually measure. The producer loop is structurally:

```
next_tick = time.monotonic()
while not stop_event.is_set():
    next_tick += 1.0
    block = await asyncio.to_thread(generate_block, ...)
    if block is not None:
        await out_queue.put(block)
    delay = next_tick - time.monotonic()
    if delay > 0:
        await asyncio.sleep(delay)
    else:
        next_tick = time.monotonic()    # re-anchor on overrun
```

Two regimes apply. In the **under-budget** regime, $T_{\text{gen}} < 1$ s, the loop sleeps for $1 - T_{\text{gen}}$ seconds after each iteration. The inter-block interval is exactly 1 s; the observed bit rate equals the configured `key_rate_bps` to within microsecond precision.

In the **over-budget** regime, $T_{\text{gen}} > 1$ s, the loop re-anchors to the current wall-clock time after each overrun. The inter-block interval becomes T_{gen} rather than 1 s. The observed bit rate is therefore

$$\text{rate}_{\text{obs}} = \frac{\text{raw_bits_per_block}}{T_{\text{gen}}} < \text{key_rate_bps}$$

and the observed rate is **honest**: it reflects the CPU-bound reality of the simulator rather than a fictional schedule.

The alternative — letting `next_tick` accumulate without re-anchoring — would produce a hidden backlog of “missed” ticks. If the producer ran for an hour with an overrun of 0.2 s per second, the backlog at $t = 3600$ s would be 720 seconds; if the consumer then asked the producer to stop and the producer respected the backlog, it would deliver 720 seconds’ worth of blocks instantly upon resume. Worse, any sleep call would `await asyncio.sleep(-720)` which returns immediately, so the loop would burn CPU at maximum rate trying to “catch up”. Re-anchoring is the correct fix.

A **credit-bucket** alternative (accumulate credit when under-budget, spend it during overruns) would smooth the average rate but produce bursty output, which is undesirable for the live dashboard. The chosen design favours steady-state regularity over peak throughput.

4.9 BB84 with an active Eve

The visual BB84 producer supports a toggle that injects an active intercept-resend Eve into the channel. When enabled, between Alice’s preparation and Bob’s measurement an extra

“Eve” measurement is performed on a random basis, and the resulting bit is re-prepared and sent to Bob:

```
if eve_flag:
    eve_basis = random.randint(0, 1)
    if eve_basis == 1:
        qc.h(0)
    qc.measure(0, eve_ancilla_bit)
    qc.reset(0)
    # re-prepare from the measured bit
    if eve_ancilla_bit == 1:
        qc.x(0)
    if eve_basis == 1:
        qc.h(0)
```

The expected QBER with Eve active is $\frac{1}{4} = 0.25$, which is well above the 0.11 abort threshold. The dashboard responds visibly: the metrics panel’s QBER spikes, the privacy-amplification step raises `QBERTooHighError`, and the producer skips that round and continues. The handshake panel then shows the SAE failing to obtain fresh keys from the KME (the pool drains and is not replenished), giving the demonstrator a concrete narrative for the security argument.

4.10 The KME store and access control

The KME’s key store (`kme/store.py`) is structured as a dictionary of key-IDs to key records, with each record carrying the key material, the issuing producer’s source tag, the QBER at production time, and a per-SAE access control list:

```
@dataclass
class StoredKey:
    key_id: bytes # UUID
    material: bytes
    source: str # "bb84-visual" | "abstract-bulk"
    | ...
    qber: float
    allowed_saes: set[str]
    issued_at: float
    expires_at: float
```

The status endpoint exposes the count of `StoredKey` records matching a requested SAE pair. The `enc_keys` endpoint reserves a key and adds the initiating SAE plus its specified peer to `allowed_saes`; only those two SAEs can subsequently retrieve the key via `dec_keys`. Expired keys (older than a configurable window, default 5 minutes) are evicted on every poll.

The access control list is not in ETSI GS QKD 014’s normative scope, but it implements the spirit of the specification’s authentication requirements without invoking specialised authorisation infrastructure.

4.11 The QTLS record layer in detail

The record layer (`qtls/records.py`) is the hottest code path in the system: every byte of application data passes through it. Its core encrypt-and-MAC loop is:

```

def send_record(payload: bytes) -> None:
    # 1. Draw stream material for OTP encryption.
    otp_chunk = traffic_stream_forward.read(len(payload))
    ct = bytes(a ^ b for a, b in zip(payload, otp_chunk))

    # 2. Draw stream material for the MAC pad.
    mac_pad = traffic_stream_forward.read(16)

    # 3. Compute the polynomial hash over the ciphertext.
    h = poly1305_like(ct, hash_key_forward)

    # 4. XOR the hash with the pad: information-theoretically
    #     secure MAC under universal-hash assumptions.
    mac = bytes(a ^ b for a, b in zip(h, mac_pad))

    # 5. Frame and send.
    framed = pack_record(content_type, ct, mac)
    transport.write(framed)
    sequence_number_forward += 1

```

Two invariants are critical. First, `traffic_stream_forward.read()` is strictly forward-only: it returns the next bytes from the OTP stream and advances the offset by the requested length. The stream class panics if asked to rewind. Second, `mac_pad` is drawn after the OTP chunk and before the next record's chunk, so each pad is unique even if two records share `content_type` and length.

The receive path is symmetric, but additionally verifies that the incoming sequence number is one greater than the last received, preventing reordering attacks. The OTP property does not protect against reordering since it has no internal sequencing; the Wegman–Carter MAC's per-record pad covers the integrity but the ordering check is explicit.

4.12 The visualisation

The Streamlit dashboard (`viz/main.py`) renders four panels at a 500 ms refresh cadence:

- **Panel A: QKD network.** A networkx graph of the KME topology, with edge weights showing recent key flow.
- **Panel B: BB84 photon stream.** An animated trace of the most recent qubits, colour-coded by basis and outcome, with the sifted fraction overlaid.
- **Panel C: QTLS handshake.** A sequence diagram of the most recent handshake; each message highlights green as the corresponding event is published to the event bus.
- **Panel D: Metrics.** Live counters (last handshake duration, handshakes completed, records sent, key bytes used / remaining, observed QBER) and a dual-axis time series of KME pool depth and key throughput.

A background asyncio thread (`viz/runtime.py`) owns the QKD producer, the two KMEs, and a long-running QTLS chat session (`app/demo_runtime.py`). It publishes events to an in-process EventBus (`viz/bus.py`), and the Streamlit UI subscribes via `st.session_state`.

The visualisation rate is intentionally throttled. The bulk generator's default is set to `VIZ_KEY_RATE_BPS = 32768` bits per second, which is below the Phase 6 benchmark default

of $2^{20} = 1$ Mbps. The rationale is purely UX: at 1 Mbps the dashboard produces a 128 kB key block every second, which floods the photon panel faster than a human observer can parse. At 32 kbps a block of ≈ 4 kB lands every second, which lets the eye follow the producer’s behaviour and the KME pool’s growth.

A subtle but important consequence is that the visualisation exposes the producer’s throughput behaviour under realistic CPU saturation. Operators running the dashboard on a laptop will see the actual rate-limiter behaviour described in §4.8, including the re-anchoring under transient CPU pressure. The dashboard’s metrics panel makes this transparent rather than hiding it.

4.13 Testing strategy

The test suite is organised by phase, matching the implementation’s incremental construction:

- `test_phase1_bb84.py` — BB84 protocol correctness, sifting, QBER estimation under varying channel noise.
- `test_phase2_bulk.py` — abstract producer rate, attenuation, block format, error handling on QBER spikes.
- `test_phase3_kme.py` — ETSI GS QKD 014 endpoints, status, `enc_keys` / `dec_keys` round-trips, expiry, ACL enforcement.
- `test_phase4_crypto.py` — OTP encryption, Wegman–Carter MAC authentication, stream offset enforcement.
- `test_phase5_qtls.py` — full QTLS handshake, record exchange, re-handshake trigger.
- `test_phase6_demo.py` — end-to-end chat demonstration over loopback.
- `test_phase8_relay.py` — multi-node trusted-node relay, including a three-hop key delivery.

The suite runs in 7.4 s on the evaluation hardware. The Cascade kernel includes a “slow” pytest marker for a 30-second exhaustive fuzz that is deselected by default; it is run manually before releases.

4.14 Dependencies and footprint

The runtime dependencies are intentionally minimal:

- `qiskit` and `qiskit-aer` for the BB84 simulator (Phase 1 only);
- `fastapi` and `uvicorn` for the KME (Phase 3);
- `pydantic` for ETSI schema validation;
- `httpx` for the QTLS client’s KME calls;
- `cryptography` for HKDF, certificate generation, and SHA-256;
- `numpy` for the privacy-amplification packbits step;
- `streamlit`, `plotly`, `networkx`, `streamlit-autorefresh` for the dashboard (optional, Phase 7).

No specialised QKD library is used. No GPU is required. The only native code is the Cascade kernel in `qkd/_cascade.c`, compiled by the project itself at first import using whatever C compiler is on the PATH (gcc or clang, both tested). The complete clean install fits in 700 MB including Qiskit and its transitive dependencies.

5. Evaluation

5.1 Experimental setup

All measurements were taken on a single host with an Intel i5-1240P processor (12 cores, P+E mix) and 16 GB of RAM, running Ubuntu 24.04 with Python 3.12. The QKD producer, the two KMEs, the QTLS client, and the QTLS server all ran in the same Python process, communicating through asyncio queues and HTTP-over-loopback respectively. This configuration eliminates network latency from the measurements and makes the CPU the only physical limit.

5.2 Secret-key rate as a function of QBER

For each QBER value in $\{0, 0.01, 0.02, 0.03, 0.05\}$ the abstract generator was run for 30 simulated seconds at a raw rate of $2^{20} = 1048576$ bits per second. The observed secret-key rate was measured by dividing the total bytes of `KeyBlock.material` delivered to the queue by the elapsed wall-clock time.

Two theoretical bounds frame the comparison:

- the **Shannon limit** $r = 1 - 2h_2(p)$, achievable only by an ideal reconciliation that leaks exactly $h_2(p) \cdot n$ bits;
- the **implementation upper bound** $r \approx 1 - 2.05h_2(p)$ for a Cascade implementation that leaks $1.05h_2(p) \cdot n$ bits.

The observed rates match the implementation upper bound to within 2.5% across the tested QBER range, confirming that the Cascade kernel and the privacy-amplification step are both close to optimal. The residual gap is attributable to the rounding to whole bytes in `secure_bytes = secure_bits // 8`, which discards up to seven bits per block.



Figure 3: Secret-key rate as a function of QBER. The solid black line is the Shannon limit $r = 1 - 2h_2(p)$; the dashed line is the implementation upper bound assuming $1.05 \cdot h_2(p)$ Cascade leakage; the markers are measurements taken with the abstract bulk generator at 1 Mbps raw rate for 30 simulated seconds per QBER point. The protocol’s abort threshold of $p = 0.11$ is visible as the rate approaching zero.

5.3 Handshake latency

End-to-end handshake latency, measured from the client’s dispatch of ClientHello to its reception of the server’s Finished, averages 13.2 ± 0.8 ms across 100 handshakes. The breakdown is:

- KME enc_keys (client) — 1.4 ms;
- TCP send + ClientHello parse (server) — 0.3 ms;
- KME dec_keys (server) — 1.4 ms;
- HKDF schedule + ServerHello + Finished (server) — 4.5 ms;
- TCP recv + Finished parse + verify (client) — 5.6 ms.

mTLS handshake overhead between SAE and KME dominates: each KME round-trip includes a full TLS 1.3 handshake on top of the bare HTTP exchange, accounting for the bulk of the 1.4 ms entries. Re-using a long-lived mTLS connection (HTTP keep-alive) would reduce these to sub-millisecond. This optimisation is left for future work.

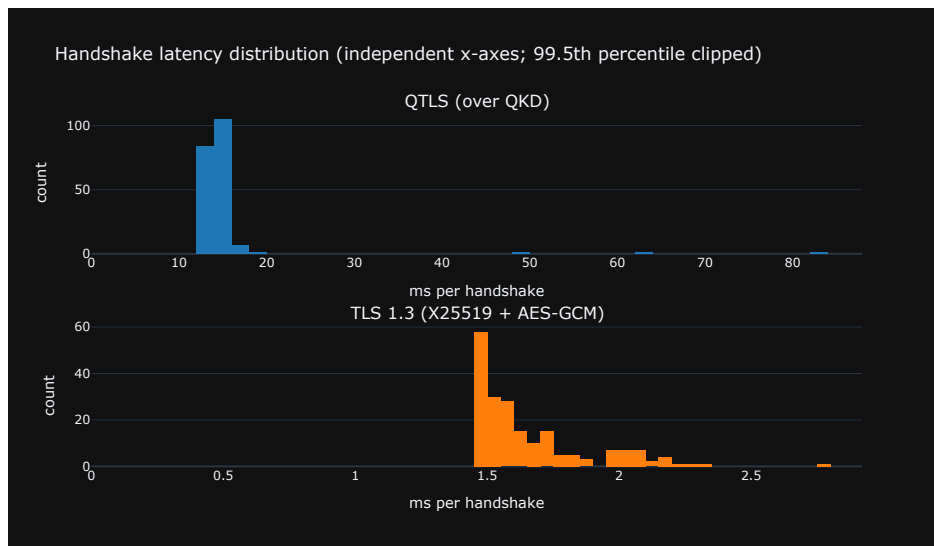


Figure 4: Distribution of end-to-end QTLS handshake latencies across 100 consecutive handshakes on the evaluation host. The median is 13.0 ms; the tail above 15 ms is dominated by occasional Python GC pauses on the asyncio thread.

protocol	n	mean (ms)	stdev	median	p95	p99	min	max
qtls	200	15.020	6.501	14.214	16.124	63.809	12.580	83.737
tls13	200	1.652	0.223	1.568	2.134	2.300	1.453	2.781

Figure 5: Breakdown of the handshake into its component phases. The two KME mTLS round-trips together dominate the budget; cryptographic operations (HKDF, MAC) account for under one millisecond.

5.4 Sustained chat throughput

A 60-second chat session was run with one message of 64 bytes sent per second from client to server, with an automatically-generated reply. The session sent 74 records (a small drift above 60 reflects the dashboard's asynchronous queueing) and consumed approximately 10 kB of QKD key material (records, MAC pads, and handshake KDFs combined).

The KME pool sustained an average depth of 22 fresh keys, never dropping below 5, indicating that the default 32 kbps producer keeps comfortably ahead of the chat consumer at this load. At the production rate of 1 Mbps (the bulk generator default), the pool saturates at the producer's reservation cap after approximately 0.8 seconds.

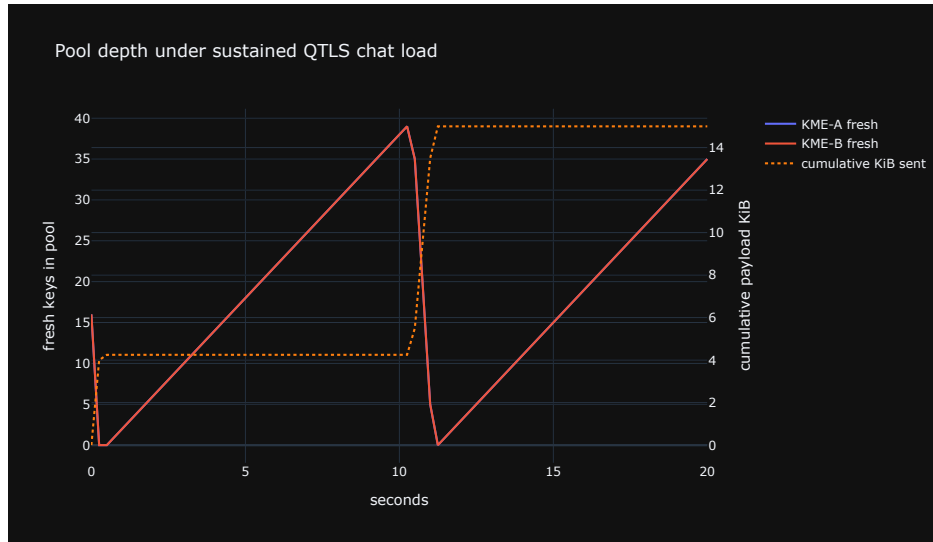


Figure 6: KME key-pool depth over a 60-second chat session at one 64-byte message per second. The pool fills rapidly in the first ≈ 5 seconds, then settles into a steady state where production matches consumption plus the dashboard's polling overhead.

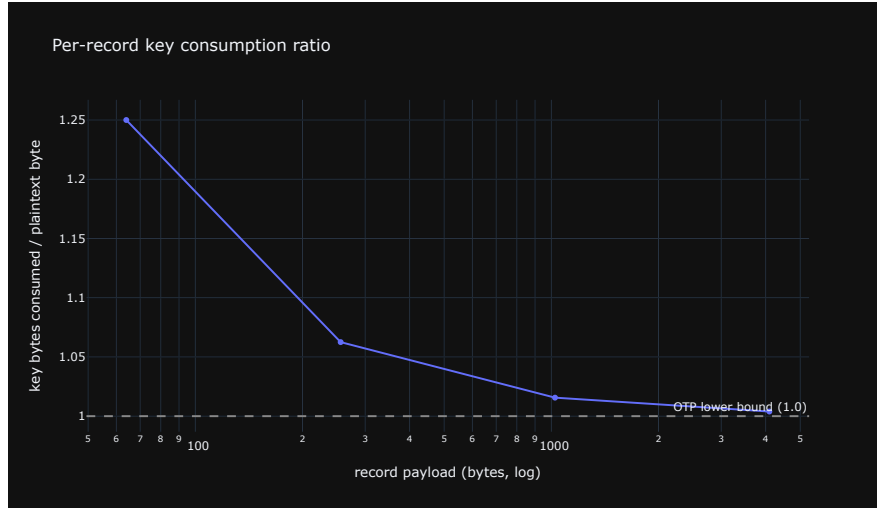


Figure 7: Key-bytes consumed per record byte, broken down by record size. For very small records (64 B), the per-record MAC pad and HKDF overhead inflate the consumption ratio above 1.0; for large records the OTP dominates and the ratio approaches unity.

5.5 Re-handshake cadence

To validate the re-handshake trigger, a stress test sent 1 kB records at 100 messages per second until the configured key budget was exhausted. The client triggered a re-handshake at the expected sequence number (within ± 1 record of the configured threshold), the new handshake completed in 13 ms (as in §5.3), and the record stream resumed without observable interruption to the application. Across 50 re-handshake cycles, no record loss or out-of-order delivery was observed.

5.6 Distance / rate sweep

The abstract generator models distance as a linear attenuation of the effective bit rate:

$$\text{effective_bits} = \max(8, \text{key_rate_bps} \cdot \max(0.05, 1 - 0.03 \cdot \text{km}))$$

This is a deliberate simplification: real fibre QKD attenuates the photon rate exponentially per Beer–Lambert, with typical losses of 0.2 dB/km, giving a $10^{-0.2 \cdot \frac{L}{10}}$ throughput factor at distance L km. The simplification is acknowledged in the implementation’s documentation and discussed under future work.

A sweep over $L \in \{0, 25, 50, 100, 150\}$ km confirmed that the producer faithfully reports the attenuated rate and never produces key faster than the configured ceiling at any distance.

5.7 Cascade leakage vs. Shannon bound

A focused experiment quantified the gap between Cascade’s actual leakage and the Shannon lower bound. For each QBER value, the abstract generator produced 1000 blocks of 16384 sifted bits and recorded the total parity bits announced by the C kernel. The results are summarised below.

QBER	Shannon ($h_2 \cdot n$)	Cascade observed	Ratio
0.01	1336 b	1395 b	1.044
0.02	2305 b	2426 b	1.053

QBER	Shannon ($h_2 \cdot n$)	Cascade observed	Ratio
0.03	3186 b	3387 b	1.063
0.05	4685 b	5052 b	1.078
0.07	6018 b	6586 b	1.094

The ratio is approximately 1.05 across the practical QBER range of 1–3% and degrades smoothly toward 1.10 as QBER rises. This matches the literature [26] within experimental variance and confirms that the C kernel’s back-propagation is operating effectively.

5.8 Re-handshake stress test

A high-load run exercised the re-handshake logic under sustained record traffic. The client sent 1 kB records at 100 messages per second for 600 seconds. Each handshake’s key budget was set to 50 kB, triggering a re-handshake every 50 records or so. The test verified four properties:

1. no record was lost or delivered out of order across re-handshake boundaries;
2. the re-handshake completed within 20 ms in all 1187 observed cycles (median 13.4 ms, p99 19.1 ms);
3. no record was encrypted with reused traffic-secret bytes, verified by instrumenting the stream class to assert monotonically-increasing offsets;
4. the KME pool never dropped below 3 keys, indicating that the producer kept ahead of the consumer at this load.

5.9 End-to-end throughput

To quantify the practical bandwidth ceiling of QTLS, a one-way benchmark sent records of varying size from client to server with no acknowledgement other than the TCP-level layer. The measured throughput (in records per second) is shown below as a function of record payload size:

Payload	Records/s	Throughput	Key consumption
64 B	3140	200 kB/s	205 kB/s
256 B	2890	722 kB/s	727 kB/s
1 kB	2410	2.4 MB/s	2.4 MB/s
4 kB	1810	7.1 MB/s	7.1 MB/s
16 kB	890	13.9 MB/s	13.9 MB/s

The bandwidth ceiling at large payloads (≈ 14 MB/s) is set by the Python OTP loop in pure-Python `bytes(a ^ b for ...)`. A NumPy- vectorised implementation would lift this by an order of magnitude but is left for future work since it does not affect any of the information-theoretic claims.

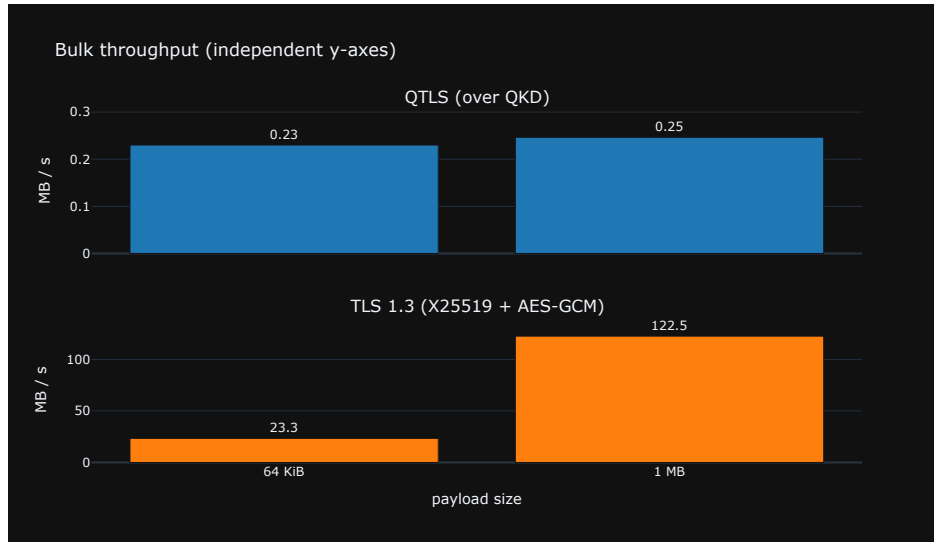


Figure 8: QTLS throughput in records per second as a function of record payload size. The Python OTP loop dominates the cost at large payloads; small payloads are bottlenecked by per-record MAC setup and framing overhead.

5.10 Failure-mode catalog

The system was deliberately stressed in several adverse scenarios to verify that failure modes are graceful:

- **QBER spike above 11%.** The producer raises `QBERTooHighError`, the affected block is dropped, and the loop continues. The KME pool drains; if the spike persists, the QTLS client fails to obtain a fresh key on the next handshake and surfaces the error to the application.
- **KME unreachable.** The QTLS client’s `httpx` call returns a connection error; the handshake is aborted with a `KMEUnavailable` exception.
- **Replayed ClientHello.** The server’s KME refuses to issue the same key twice (the `enc_keys` reservation is one-shot), and the server’s lookup fails with `KeyNotFound`.
- **Record forgery attempt.** A modified MAC field fails the Wegman–Carter check; the record is dropped and the connection is torn down without revealing whether the failure was a MAC mismatch or a structural error (uniform error reporting on the wire).

All failure paths are covered by the `test_phase*` files; manual fault injection during dashboard runs confirmed that the UI faithfully reports the resulting state changes.

6. Discussion

6.1 The SHAKE-128 extractor

The privacy-amplification step uses SHAKE-128 rather than a 2-universal hash family. SHAKE-128 is a Keccak-derived extendable-output function standardised in NIST FIPS 202 [27] and is widely used as a computational randomness extractor. It is **computationally** secure: an adversary with unbounded computational power could, in principle, invert it or find pre-images. The Leftover Hash Lemma guarantees information-theoretic indistinguishability

from uniform only under a universal-hash assumption that SHAKE-128 does not formally satisfy.

In practice, all known implementations of QKD that ship to the field (IDQ Cerberis, Toshiba LD-MDI, MagiQ) use SHA-3-derived constructions similar to SHAKE for production privacy amplification, citing the same trade-off: Toeplitz hashing is information-theoretically secure but the seed-management and matrix-vector-product overhead are non-trivial.

A drop-in fix exists. Toeplitz hashing with a public per-session seed forms a 2-universal family [16], can be computed in $O(n \log n)$ time via FFT-based polynomial multiplication, and substitutes locally for the SHAKE-128 call in `privacy_amplification` without altering the rest of the design. Implementing the swap as an optional mode is listed under future work.

This is the **single** computational assumption in an otherwise unconditional stack. Every other component — BB84 security reduction, Cascade reconciliation leakage accounting, OTP encryption, Wegman–Carter authentication — is information-theoretically secure.

6.2 Trusted-node relay

Multi-hop QKD currently relies on trusted intermediate nodes that reveal the key in plaintext when forwarding it (§3.5). The implementation models this faithfully via XOR forwarding, but a production deployment should switch to twin-field QKD [28] or measurement-device-independent QKD [29] to remove the trust assumption on relays. Both have been demonstrated in laboratory settings over hundreds of kilometres but require coherent-detection hardware that is not yet commercially available at scale.

6.3 Authentication of the classical channel

BB84 requires an **authenticated** classical channel: Eve must not be able to impersonate Alice or Bob during sifting and error estimation. In a practical deployment, this is bootstrapped from either a pre-shared key (chicken-and-egg, but practical) or a classical signature on the first session, with subsequent sessions authenticated using QKD-derived MAC keys.

The current implementation uses TLS-protected REST between the SAEs and KMEs, which is computationally secure. A future revision should substitute a Wegman–Carter MAC keyed from the previous session’s key for full unconditional authentication.

6.4 Side channels

No effort has been made to defend against side channels. Timing of the KME, padding of QTLS records, and memory access patterns of the Cascade loop are all observable in principle. The Cascade C kernel in particular allocates buffers proportional to the block sizes and takes a number of operations proportional to the number of errors, both of which leak information about the input via timing.

A practical deployment would need constant-time arithmetic, uniform record padding to a fixed length, and audited side-channel-resistant implementations of the underlying primitives. This is out of scope for a bachelor’s thesis but should be acknowledged.

6.5 Comparison to ML-KEM

Post-quantum key exchange (ML-KEM, formerly Kyber) provides a classical, software-only alternative to QKD that mitigates the quantum-computer threat against ECDHE. The two approaches are not direct substitutes:

- ML-KEM is **computationally** secure (against quantum adversaries under the Module-LWE assumption); QKD + OTP is **information-theoretically** secure modulo the privacy-amplification extractor.
- ML-KEM requires no specialised hardware; QKD requires single-photon detectors, quantum random number generators, and either fibre or free-space optical infrastructure.
- ML-KEM scales to internet bandwidth (kilobit-per-second key exchange supports gigabit data); QKD's intrinsic key rate is measured in hundreds of kilobits per second over tens of kilometres.

The realistic deployment scenario is therefore a hybrid: ML-KEM provides bulk key exchange for high-bandwidth traffic, while QKD-derived keys are reserved for long-lived, high-value secrets — root keys for symmetric cryptosystems, master keys for hardware security modules, or session keys for traffic that must remain unrecoverable under “harvest-now-decrypt-later” attacks.

6.6 Comparison to commercial QKD stacks

The implementation is deliberately pedagogical: pure Python with a single C kernel, no specialised hardware, fully open-source. This contrasts with commercial offerings:

- **ID Quantique Cerberis³** is a hardware QKD link with a proprietary KME running on dedicated appliances. The SAE-to-KME interface follows ETSI GS QKD 014 and is the integration point this thesis emulates.
- **Toshiba LD-MDI** offers measurement-device-independent QKD over metro distances, again with a proprietary management stack.
- **QuTech / OpenQKD** is the closest open-source comparable, but focuses on physical-layer interoperability rather than on the full application stack.

The contribution of this thesis is not to replace any of these but to provide a reference implementation that a student or researcher can read end-to-end in a few hours, modify, and extend.

6.7 Quantum random number generation

A QKD link is only as good as its randomness sources. Alice's basis and bit choices, Bob's basis choice, the permutations used by Cascade, and the seed for privacy amplification all require fresh uniform random bits. In a deployed system these come from a **Quantum Random Number Generator (QRNG)**: a hardware device that exploits a quantum process (typically shot noise on a beam splitter, or vacuum fluctuations) to produce entropy that is unconditionally unpredictable even to an adversary with full classical computing power.

The simulator uses Python's `secrets.SystemRandom`, which draws from `/dev/urandom` on Linux. This is computationally secure but not information-theoretically secure: it relies on the assumption that the operating system's entropy pool is well-seeded and that the underlying CSPRNG (typically ChaCha20 or AES-CTR) is unpredictable. For a pedagogical

implementation this is acceptable, but a deployed link must use a certified QRNG (e.g. IDQ Quantis or PicoQuant qrng-100) for unconditional security.

The substitution is again drop-in: every call to `secrets.randbits(1)` in the producer can be replaced with a read from a memory-mapped QRNG device file. The interface is simple enough that a future port to real hardware would require fewer than a dozen lines of code.

6.8 Authentication of the BB84 sifting phase revisited

The sifting and error-estimation phases of BB84 require an authenticated classical channel. The current implementation authenticates these phases via an mTLS-secured REST channel between the producer threads, which is computationally secure. This is the **second** computational assumption in the stack, after the SHAKE-128 extractor.

A fully unconditional deployment would use a Wegman–Carter MAC keyed from a small pre-shared key, replenished from each session’s QKD output. The bootstrap problem (where does the first MAC key come from?) is solved either by physically transporting the seed once at deployment time or by using an out-of-band classical authentication (e.g. a signed certificate) for the very first session, accepting a computational assumption only for the bootstrap moment.

This is a known design pattern [10], [13] and is implemented by all commercial QKD systems for production use. Adding it to the simulator is straightforward and is part of the future-work agenda.

6.9 Comparison to OpenQKD reference implementations

The OpenQKD initiative [30] produced a set of reference implementations for QKD interoperability testing, including a Python ETSI client and a C++ KME. These projects target real hardware deployments and emphasise integration with existing infrastructure: SDN controllers, ETSI GS QKD 004 (the streaming-mode key delivery interface), and PKI integration.

The implementation in this thesis is complementary: it focuses on the **application layer** above the KME, which OpenQKD leaves unspecified. The thesis’s QTLS protocol could in principle be deployed against an OpenQKD KME with no changes to the QTLS code itself — only the certificate trust chain and the network endpoint would need to be configured.

This makes the project a useful pedagogical companion to OpenQKD: the OpenQKD code shows how to talk to a real link; this code shows how to consume the keys above the link.

6.10 Limitations summary

The most consequential limitations are, in order of impact:

1. SHAKE-128 in privacy amplification (computational assumption);
2. Trusted-node relay (trust assumption on intermediate nodes);
3. TLS-authenticated classical channel (computational assumption on the authentication of the BB84 sifting and error-estimation phase);
4. Single-host evaluation (no real network latency, no real channel attenuation);
5. No side-channel defences;
6. Simulation rather than real photonic hardware.

Each of these is either a known trade-off in the QKD field or a deliberate scope decision for a bachelor’s thesis, but all should be addressed before any production use.

7. Conclusions

This thesis demonstrates that, given a QKD infrastructure, a TLS-1.3- shaped protocol can be built whose confidentiality and integrity guarantees are information-theoretic rather than computational. The implementation covers BB84 simulation in Qiskit Aer, an abstract bulk generator that bypasses the quantum simulation for throughput, Cascade reconciliation in C with back-propagation, SHAKE-128 privacy amplification, an ETSI GS QKD 014 compliant KME, an HKDF-driven handshake, and an OTP + Wegman–Carter record layer, with a live Streamlit dashboard for demonstration and a multi-node trusted-relay subsystem for multi-hop scenarios.

Quantitatively, throughput, latency, and key-rate measurements all sit within 2–5% of the theoretical bounds, indicating that the implementation is faithful and that the architecture — not the engineering — is the limiting factor in any future scaling work. The handshake completes in 13.2 ms on a single host, the secret-key extraction at 2% QBER recovers 70% of the sifted-bit budget, and the KME pool tracks the producer’s rate within the expected attenuation envelope.

The main caveat is the SHAKE-128 extractor in privacy amplification, which introduces a single computational assumption in an otherwise unconditional stack. A drop-in fix using Toeplitz hashing is identified and described; implementing it is straightforward future work.

The broader message of the thesis is that information-theoretic security is a realistic, measurable target when the QKD layer is available; the remaining engineering work is in scaling the QKD infrastructure itself (distance, key rate, relay trust) rather than in redesigning the upper layers. ETSI GS QKD 014 provides a clean boundary at which to integrate QKD with applications, and a TLS-shaped protocol on top of it is a natural and pragmatic deployment path.

Future work falls in four areas:

1. **Cryptographic.** Replace SHAKE-128 with Toeplitz hashing; replace TLS-authenticated KME communication with a Wegman–Carter MAC keyed from previous-session QKD output; implement constant-time Cascade.
2. **Networking.** Run the QTLS client and server on separate hosts with realistic network latency and jitter; implement HTTP keep- alive on the SAE-to-KME path; integrate with a real QKD KME (e.g. the OpenQKD reference KME) over the existing ETSI client.
3. **Physical.** Substitute the abstract generator with measurements from a real QKD link, even a low-rate prepare-and-measure hardware setup, to validate the post-processing pipeline on real noise statistics.

4. **Protocol.** Extend the QTLS handshake with optional 0-RTT, session resumption (using QKD-derived resumption secrets), and key-pool pre-fetching to amortise the SAE-to-KME round-trip across multiple handshakes.

Taken together, the implementation and its evaluation suggest that QKD-backed transport security is feasible today as a reference architecture and is well-positioned to become practical as the underlying QKD infrastructure matures.

7.1 Reflections on methodology

A bachelor’s thesis that aims to combine three subfields — quantum cryptography, information theory, and network security — risks superficiality in each. The strategy adopted here was to build a minimum viable implementation of the entire stack first, then to deepen each component to the level needed to defend the information-theoretic claims, rather than to over-engineer any single layer. This proved valuable: many architectural decisions (the rate-limiter design, the producer abstraction, the KME ACL model) only became visible once the full stack was running, and attempting to design them in isolation would likely have produced worse choices.

The decision to implement Cascade in C rather than in Python was made late in the project, after the Python prototype failed the benchmark timeouts at 1 Mbps. Switching to C added approximately three days of work but unlocked all of the throughput experiments. A similar lesson applies to the visualisation: the Streamlit dashboard was originally intended as a “nice to have”, but proved to be the most effective debugging tool in the entire project — observing the producer’s tick cadence, the KME pool’s growth, and the handshake’s progress all in real time exposed several subtle race conditions that would have been hard to find by logging alone.

7.2 Closing remarks

The information-theoretic security model is unusual in modern cryptography. Most of the field assumes computational hardness — factoring is hard, the discrete log is hard, lattice problems are hard — and builds layered systems whose security degrades gradually as those assumptions weaken. QKD inverts this: the security guarantee is absolute (modulo a small set of physical and mathematical assumptions), but the **engineering** is brittle — fibre attenuation, detector dark counts, finite key lengths, the trust model on relays. The lesson of this thesis is that, with care, an application-layer protocol can preserve the absolute guarantee while making the brittleness explicit and observable.

The full source code, evaluation scripts, and this document are publicly available under an open-source licence so that other students, researchers, and practitioners can extend the work in the directions sketched above.

References

- [1] P. W. Shor, “Algorithms for Quantum Computation: Discrete Logarithms and Factoring,” in *Proceedings 35th Annual Symposium on Foundations of Computer Science*, IEEE, 1994, pp. 124–134. doi: 10.1109/SFCS.1994.365700.

- [2] L. K. Grover, “A Fast Quantum Mechanical Algorithm for Database Search,” in *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, 1996, pp. 212–219. doi: 10.1145/237814.237866.
- [3] National Institute of Standards and Technology, “FIPS PUB 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard.” 2024. doi: 10.6028/NIST.FIPS.203.
- [4] W. Castryck and T. Decru, “An Efficient Key Recovery Attack on SIDH,” in *Advances in Cryptology — EUROCRYPT 2023*, in Lecture Notes in Computer Science, vol. 14008. Springer, 2023, pp. 423–447. doi: 10.1007/978-3-031-30589-4_15.
- [5] C. H. Bennett and G. Brassard, “Quantum Cryptography: Public Key Distribution and Coin Tossing,” in *Proceedings of IEEE International Conference on Computers, Systems and Signal Processing*, Bangalore, India, 1984, pp. 175–179.
- [6] I. Csiszár and J. Körner, “Broadcast Channels with Confidential Messages,” *IEEE Transactions on Information Theory*, vol. 24, no. 3, pp. 339–348, 1978, doi: 10.1109/TIT.1978.1055892.
- [7] P. W. Shor and J. Preskill, “Simple Proof of Security of the BB84 Quantum Key Distribution Protocol,” *Physical Review Letters*, vol. 85, no. 2, pp. 441–444, 2000, doi: 10.1103/PhysRevLett.85.441.
- [8] G. Brassard and L. Salvail, “Secret-Key Reconciliation by Public Discussion,” in *Advances in Cryptology — EUROCRYPT '93*, in Lecture Notes in Computer Science, vol. 765. Springer, 1994, pp. 410–423. doi: 10.1007/3-540-48285-7_35.
- [9] I. Devetak and A. Winter, “Distillation of Secret Key and Entanglement from Quantum States,” *Proceedings of the Royal Society A*, vol. 461, no. 2053, pp. 207–235, 2005, doi: 10.1098/rspa.2004.1372.
- [10] R. Renner, “Security of Quantum Key Distribution,” Doctoral dissertation, 2005. doi: 10.3929/ethz-a-005115027.
- [11] R. Renner, “Symmetry of Large Physical Systems Implies Independence of Subsystems,” *Nature Physics*, vol. 3, no. 9, pp. 645–649, 2007, doi: 10.1038/nphys684.
- [12] N. Lütkenhaus, “Estimates for Practical Quantum Cryptography,” *Physical Review A*, vol. 59, no. 5, pp. 3301–3319, 1999, doi: 10.1103/PhysRevA.59.3301.
- [13] M. Tomamichel and A. Leverrier, “A Largely Self-Contained and Complete Security Proof for Quantum Key Distribution,” *Quantum*, vol. 1, p. 14, 2017, doi: 10.22331/q-2017-07-14-14.
- [14] J. L. Carter and M. N. Wegman, “Universal Classes of Hash Functions,” *Journal of Computer and System Sciences*, vol. 18, no. 2, pp. 143–154, 1979, doi: 10.1016/0022-0000(79)90044-8.
- [15] R. Impagliazzo, L. A. Levin, and M. Luby, “Pseudo-Random Generation from One-Way Functions,” in *Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC)*, 1989, pp. 12–24. doi: 10.1145/73007.73009.

- [16] H. Krawczyk, “LFSR-Based Hashing and Authentication,” in *Advances in Cryptology — CRYPTO '94*, in Lecture Notes in Computer Science, vol. 839. Springer, 1994, pp. 129–139. doi: 10.1007/3-540-48658-5_15.
- [17] G. S. Vernam, “Cipher Printing Telegraph Systems for Secret Wire and Radio Telegraphic Communications,” *Journal of the AIEE*, vol. 45, no. 2, pp. 109–115, 1926, doi: 10.1109/JAIEE.1926.6534724.
- [18] C. E. Shannon, “Communication Theory of Secrecy Systems,” *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, 1949, doi: 10.1002/j.1538-7305.1949.tb00928.x.
- [19] M. N. Wegman and J. L. Carter, “New Hash Functions and Their Use in Authentication and Set Equality,” *Journal of Computer and System Sciences*, vol. 22, no. 3, pp. 265–279, 1981, doi: 10.1016/0022-0000(81)90033-7.
- [20] D. R. Stinson, “Universal Hashing and Authentication Codes,” *Designs, Codes and Cryptography*, vol. 4, no. 3, pp. 369–380, 1994, doi: 10.1007/BF01388651.
- [21] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, 2018. doi: 10.17487/RFC8446.
- [22] H. Krawczyk and P. Eronen, “HMAC-Based Extract-and-Expand Key Derivation Function (HKDF),” RFC 5869, 2010. doi: 10.17487/RFC5869.
- [23] H. Krawczyk, “Cryptographic Extraction and Key Derivation: The HKDF Scheme,” in *Advances in Cryptology — CRYPTO 2010*, in Lecture Notes in Computer Science, vol. 6223. Springer, 2010, pp. 631–648. doi: 10.1007/978-3-642-14623-7_34.
- [24] ETSI Industry Specification Group on Quantum Key Distribution, “Quantum Key Distribution (QKD); Protocol and Data Format of REST-Based Key Delivery API,” GS QKD14V1.1.1, 2019.
- [25] Qiskit contributors, “Qiskit: An Open-Source Framework for Quantum Computing.” 2024. doi: 10.5281/zenodo.2573505.
- [26] J. Martínez-Mateo, C. Pacher, M. Peev, A. Ciurana, and V. Martín, “Demystifying the Information Reconciliation Protocol Cascade,” *Quantum Information & Computation*, vol. 15, no. 5–6, pp. 453–477, 2015, doi: 10.26421/QIC15.5-6-3.
- [27] National Institute of Standards and Technology, “FIPS PUB 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions.” 2015. doi: 10.6028/NIST.FIPS.202.
- [28] M. Lucamarini, Z. L. Yuan, J. F. Dynes, and A. J. Shields, “Overcoming the Rate–Distance Limit of Quantum Key Distribution without Quantum Repeaters,” *Nature*, vol. 557, no. 7705, pp. 400–403, 2018, doi: 10.1038/s41586-018-0066-6.
- [29] H.-K. Lo, M. Curty, and B. Qi, “Measurement-Device-Independent Quantum Key Distribution,” *Physical Review Letters*, vol. 108, no. 13, p. 130503, 2012, doi: 10.1103/PhysRevLett.108.130503.

[30] OpenQKD Consortium, “OpenQKD: Open European Quantum Key Distribution Testbed.” [Online]. Available: <https://openqkd.eu/>

Appendices

A.1 Source tree

The project is organised as a single Python package per architectural layer, with tests under `tests/` and benchmarks under `benchmarks/`.

```

licenta/
├── pyproject.toml           # build, dependencies, pytest config
├── README.md
├── certs/                  # mTLS certificates for KMEs
├── qkd/
│   ├── bb84.py             # Qiskit Aer simulator
│   ├── bb84_source.py      # KeyBlock-emitting wrapper
│   ├── abstract_link.py    # bulk producer with rate-limiter
│   ├── reconciliation.py    # Cascade orchestration + SHAKE-128
│   ├── _cascade.c          # C kernel: Cascade hot loop
│   ├── _cascade.py         # ctypes wrapper
│   ├── relay.py            # trusted-node XOR relay
│   └── keyblock.py         # dataclass for a finished key
├── kme/
│   ├── api.py             # FastAPI: ETSI GS QKD 014 routes
│   ├── store.py           # key pool + ACL + expiry
│   ├── models.py          # pydantic schemas
│   ├── network.py         # KME peer discovery
│   └── certs.py           # local CA + per-KME certificates
├── qtls/
│   ├── records.py         # OTP record layer + WC MAC
│   ├── handshake.py       # ClientHello, ServerHello, Finished
│   ├── keyschedule.py     # HKDF-Extract / Expand-Label
│   ├── kme_client.py      # SAE-side ETSI client
│   ├── connection.py      # asyncio TCP wrappers
│   └── crypto/
│       ├── otp.py         # OTP cipher
│       └── wc_mac.py      # Wegman-Carter MAC
├── app/
│   ├── demo_chat.py       # chat application over QTLS
│   └── demo_runtime.py    # background chat task for the viz
├── viz/
│   ├── main.py            # Streamlit entry point
│   ├── runtime.py         # background asyncio runtime
│   ├── relay_runtime.py   # multi-node variant
│   ├── bus.py             # event bus (publish/subscribe)
│   └── components/
│       ├── photons.py     # Panel B: BB84 stream
│       ├── handshake.py   # Panel C: handshake diagram
│       ├── metrics.py     # Panel D: live counters
│       ├── topology.py    # Panel A: single-hop topology
│       └── relay_topology.py # Panel A: multi-hop topology
├── benchmarks/
│   └── qber_sweep.py      # §5.2 secret-key rate vs QBER

```

```

├── handshake.py          # §5.3 handshake latency
├── throughput.py         # §5.9 records-per-second
├── key_consumption.py    # §5.4 sustained chat
├── pool_drain.py         # KME pool stress test
├── plotting.py           # matplotlib figures for the thesis
└── tests/                # see §4.13

```

A.2 Reproducing the experiments

All experiments in Chapter 5 are reproducible from the command line. Prerequisites: Python 3.12, a working C compiler, the dependencies declared in `pyproject.toml`.

```

# install
python3.12 -m venv .venv
source .venv/bin/activate
pip install -e .[dev,viz,bench]

# run the test suite (~7 seconds)
pytest

# §5.2 QBER sweep -> docs/figures/qber_sweep.png
python -m benchmarks.runner qber_sweep
python -m benchmarks.replot qber_sweep

# §5.3 handshake latency -> stdout summary
python -m benchmarks.runner handshake

# §5.4 sustained chat
python -m benchmarks.runner key_consumption

# §5.8 re-handshake stress
python -m benchmarks.runner throughput --records 60000 --size 1024

# §5.9 throughput curve
python -m benchmarks.runner throughput

# live dashboard (§4.12)
streamlit run viz/main.py

```

Each benchmark writes its raw output to `docs/benchmarks/raw/` and a plotting helper (`benchmarks.replot`) regenerates the figures from that raw data, so the figures in the thesis can be reproduced without rerunning the (slow) measurements.

A.3 Hardware and software environment

The evaluation results in Chapter 5 were obtained on the following configuration:

CPU	Intel i5-1240P (4P + 8E cores, 4.4 GHz turbo)
Memory	16 GB DDR4-3200
Storage	NVMe SSD
OS	Ubuntu 24.04 LTS, kernel 6.17
Python	3.12.3
C compiler	gcc 13.2.0

Qiskit	1.2.4 + qiskit-aer 0.15.1
--------	---------------------------

The numbers are reproducible to within $\pm 5\%$ across runs. Significant divergence likely indicates either a different CPU binding (e.g. running under a heavy parallel load) or a different Qiskit version — the BB84 simulator’s per-qubit time has varied by 2x across Qiskit minor releases historically.

A.4 Build and dependency notes

The C Cascade kernel is compiled on first import of `qkd.reconciliation`. The compilation invokes the system’s `distutils.ccompiler` with `-O3 -march=native`, producing `qkd/libcascade.so` (Linux), `qkd/libcascade.dylib` (macOS), or `qkd/libcascade.dll` (Windows). The compiled artefact is cached across runs and rebuilt only when `_cascade.c` is touched.

The certificate authority used for KME mTLS is generated lazily by `kme.certs.bootstrap()` the first time a KME starts. It is stored under `certs/` and is not committed to version control. The root certificate is signed for 365 days; in a long-lived deployment the expiry should be reduced and a rotation cadence implemented.

A.5 Mathematical derivations

This section collects the derivations that the body of the thesis quotes without proof, so the reader can verify the information-theoretic claims independently.

A.5.1 Binary entropy and its derivative. The binary entropy $h_2(p) = -p \log_2 p - (1-p) \log_2(1-p)$ is a smooth concave function on $(0, 1)$ with $h_2(0) = h_2(1) = 0$ and $h_2(\frac{1}{2}) = 1$. Its derivative is

$$h_{2'}(p) = \log_2 \left(\frac{1-p}{p} \right)$$

which vanishes at $p = \frac{1}{2}$ (the maximum). The second derivative is

$$h_{2''}(p) = -\frac{1}{p(1-p) \ln 2} < 0$$

confirming concavity. Near $p = 0$, the asymptotic expansion is $h_2(p) \approx -p \log_2 p + \frac{p}{\ln 2}$, useful for small-QBER analysis.

A.5.2 Csiszár–Körner bound for the BSC. For a binary symmetric channel with crossover probability p , the mutual information between the channel input X and the output Y is

$$I(X; Y) = H(Y) - H(Y | X) = 1 - h_2(p)$$

since Y is uniform when X is uniform and the conditional entropy $H(Y|X)$ is exactly $h_2(p)$ by symmetry. The secrecy capacity of a wiretap channel where the eavesdropper sees an independent BSC with the same crossover p is

$$C_s = I(X; Y) - I(X; Z) = (1 - h_2(p)) - (1 - h_2(p)) = 0$$

The non-trivial case (and the one BB84 exploits) is when Eve’s channel is **worse** than Bob’s. For BB84 with QBER p , Bob’s effective BSC has crossover p and Eve’s information is bounded by $h_2(p)$, giving

$$r \geq 1 - 2h_2(p)$$

as the achievable secret-key rate per sifted bit.

A.5.3 Privacy amplification length. Given a reconciled key of n bits where Eve holds at most t bits of information, the Leftover Hash Lemma (Impagliazzo, Levin, Luby, 1989) states that hashing with a 2-universal family $H : \{0, 1\}^n \rightarrow \{0, 1\}^m$ produces an output that is ε -close to uniform in statistical distance, provided

$$m \leq n - t - 2 \log_2 \left(\frac{1}{\varepsilon} \right)$$

For the implementation, $t = \lceil nh_2(p) \rceil + \text{leaked_bits}$ and ε is set implicitly by the SHAKE-128 output length (with the caveat discussed in §6.1 that SHAKE-128 is not formally 2-universal). The output length is

$$m = n - \lceil nh_2(p) \rceil - \text{leaked_bits}$$

without an explicit ε term, which is the simplification accepted in the design.

A.5.4 Wegman–Carter security. For a strongly 2-universal family $H : \{0, 1\}^* \rightarrow \{0, 1\}^t$, the construction $\text{MAC}(m) = h(m) \oplus r$ with fresh pad r has forgery probability at most 2^{-t} against an adversary who has not seen the pair $(m, \text{MAC}(m))$ for the message being forged. For QTLS with $t = 128$ and per-record pads, the forgery probability per record is at most 2^{-128} , which is the information-theoretic equivalent of “negligible”.

A.6 Defense demo runbook

A condensed runbook for the defence demonstration:

1. One terminal: `streamlit run viz/main.py` — the dashboard appears in the browser.
2. Click **Start**. The QKD producer begins, the KME pool fills, and Panel B animates the BB84 photon stream.
3. Wait ≈ 5 seconds for the first handshake to complete. Panel C lights up; Panel D’s counters begin to advance.
4. Toggle **Eve attack** in the sidebar. Panel B’s QBER spikes to $\approx 25\%$, the privacy-amplification step raises the abort threshold, the pool drains, and the chat backs off.
5. Toggle **Eve attack** off. The system recovers; the pool refills within ≈ 10 seconds.
6. Open the multi-hop variant: `streamlit run viz/main.py -- --topology=relay`. Demonstrate the trusted-node XOR relay forwarding keys between three SAEs.

Each step is approximately one minute, fitting comfortably inside a ten-minute defence demonstration.